

Chapter 0 穩定配對

<Gale-Shapley>

```
while(有男生沒結婚且還沒有跟每個人求婚){
    選該男生 m;
    令 w 為 m 求婚的選擇;
    if (w 沒結婚) m 和 w 結婚;
    else if (w 已結婚但較喜歡 m 勝過目前){
        w 和目前配偶離婚, 和 m 結婚;
    }
    else w 拒絕 m;
}
```

→ 假設有 n 位男性和 n 位女性要配對
Time Complexity: $O(n^2)$ / Space Complexity: $O(n^2)$

Chapter 1 Preliminaries

屬於: $\in$ / 不屬於: $\notin$ / $A \subseteq B$: A 是 B 的子集合
(每一個在 A 的元素也在 B 中)

真子集 Proper Subsets ($\subset$): A 是 B 的子集, 但 $A \neq B$
空集合的表示: $\{\}$, $\emptyset$

集合的寫法: $\{a_1, a_2, a_3, \dots\}$, $\{N \mid \text{rule about } N\}$
全把元素寫出來 Ex. $\{n \mid n = m^2 \text{ for } m \in \mathbb{N}\}$

序列: 順序重要, 可接受重複。(表示法: 小括號)
集合: 順序不重要, 重複無意義。(表示法: 大括號)
元組: 有限長度的序列, 比如 (2, 4, 8, 6) 是 4 元組 (4-tuple)
冪集合 Power Set: 某集合所有子集所形成的集合。

- $A = \{1, 2\}$, Power Set of A: $\{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$
迪卡兒積 Cartesian Product (表示法: $\times$)
一種由集合元素形成的 Pair 集合。例: $A = \{1, 2\}$, $B = \{x, y, z\}$
 $A \times B = \{(1, x), (1, y), (1, z), (2, x), (2, y), (2, z)\}$

三大證明方法:

- 建構法 Proof by Construction: 為證明某個特定性質, 直接舉出實例。
- 反證法 Proof by Contradiction: 先假設結論不成立, 推導矛盾。
- 歸納法 Proof by Induction: 從個別案例中推導出一般性。

Chapter 2 Insertion Sort, Merge Sort

Insertion Sort 插入排序法 $O(n^2)$ · Stable · In-Place
從第 2 項開始排, ~ n 項止。每次都兩兩交換, 直到遇到 $\leq$ 自己的或全部排完。

Insertion-sort(A)
for $j \leftarrow 2$ to length(A)
 key $\leftarrow A[j]$

把第 j 個放到適合的位子。
 $i \leftarrow j-1$ % 從前一張開始問
 while $i > 0$ && $A[i] > \text{key}$
 do $A[i+1] \leftarrow A[i]$
 $i \leftarrow i-1$
 $A[i+1] \leftarrow \text{key}$ % 前面沒人了, 沒有比較大的

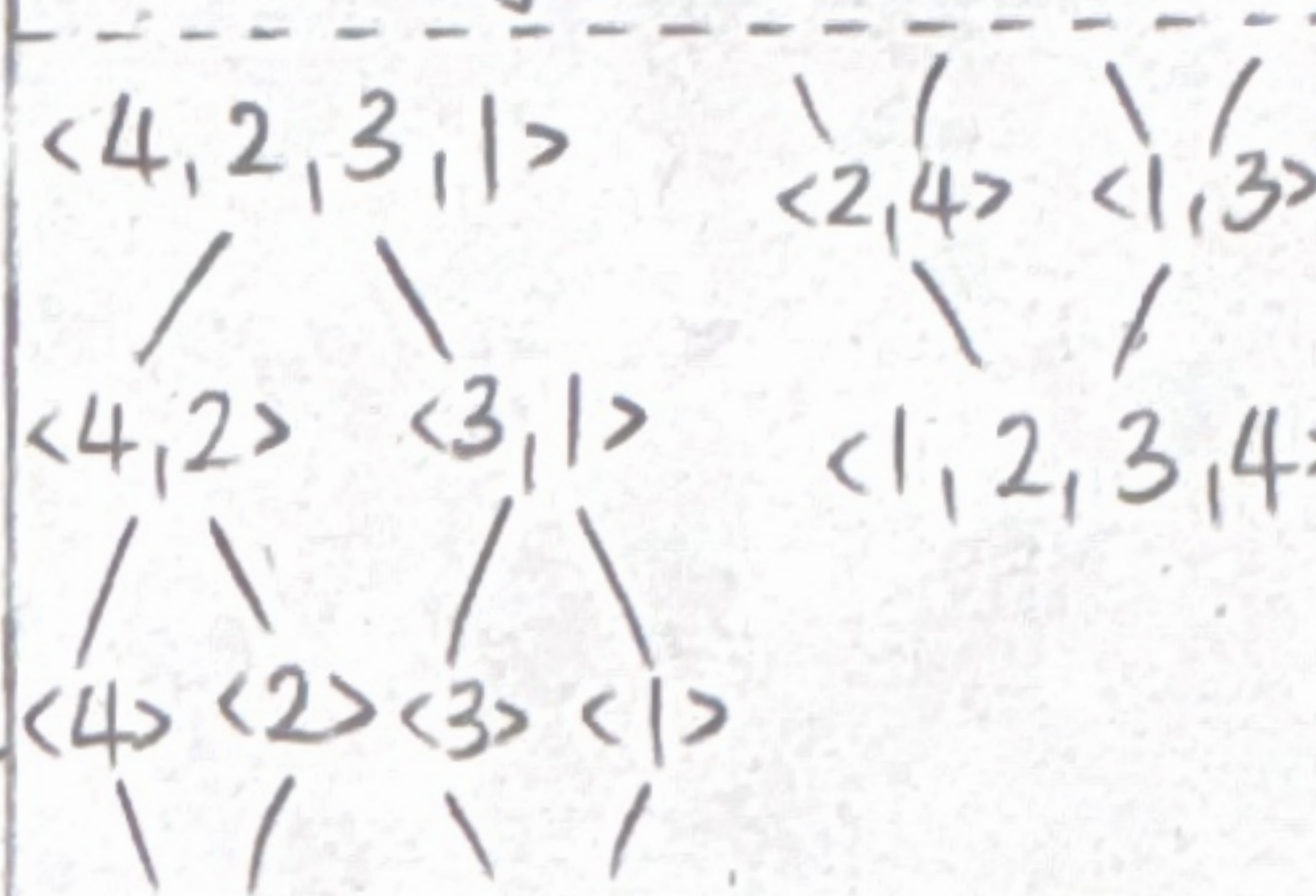
利用 loop-Invariants 證明正確性
Loop-Invariants: 做第 j 次 loop 前, 前 j-1 個元素已排好。
Initialization: $j=2$, 只有一個元素, 本身就已排好。
Maintenance: 在 loop 中, 將 $A[j]$ 與 $A[j-1]$ 往右移, 直到發現 $A[i] \leq A[j]$, 放置 $A[j]$ 。
Termination: loop 結束後, $j=n+1$, 根據不變量, $A[1 \sim n]$ 已排好。

Merge Sort 合併排序法 $O(n \log n)$ · Stable · 非 In-Place

將陣列一直拆分直至剩下一個元素, 先排左邊的子陣列, 再排右邊的, 最後向上合併回去。

merge-sort(A, l, r)
if $l < r$ % 陣列元素多於 1
 then $\text{mid} = \lfloor (l+r)/2 \rfloor$
 merge-sort(A, l, mid)
 merge-sort(A, mid+1, r)
 merge(A, l, mid, r)

merge(A, l, m, r)
 $n_1 \leftarrow m-l+1$, $n_2 \leftarrow r-m$
new arr $L(n_1+1)$, $R(n_2+1)$
for $i \leftarrow 1 \sim n_1$, $L[i] = A[l+i-1]$
for $i \leftarrow 1 \sim n_2$, $R[i] = A[m+i]$
 $L[\text{end}] \leftarrow \infty$, $R[\text{end}] \leftarrow \infty$
 $p \leftarrow 1$, $q \leftarrow 1$
for $k \leftarrow l$ to r
 if $L[p] \leq R[q]$
 $A[k] \leftarrow L[p]$
 $p \leftarrow p+1$
 else
 $A[k] \leftarrow R[q]$
 $q \leftarrow q+1$



Loop Invariants 迴圈不變量: 證明程式、演算法的正確性
證明 (I) Initialization: 在第一次 loop 執行前正確。
敘述 (II) Maintenance: loop 在第 i 次前正確, 則在第 i 次後也正確。
是對的 (III) Termination: loop 結束後, 可用 invariants 的性質證明正確性。
→ 用敘述去證明演算法是對的。

Chapter 3 θ, O, Ω Growth Functions

- 對於 $f(n)$
- $\theta(g(n))$: 滿足 $0 < c_1 g(n) \leq f(n) \leq c_2 g(n)$, $\forall n \geq n_0$ (約常數倍)
- $O(g(n))$: 滿足 $0 \leq f(n) \leq c g(n)$, $\forall n \geq n_0$ (上界) ($\leq$ 常數倍)
- $\Omega(g(n))$: 滿足 $0 < c g(n) \leq f(n)$, $\forall n \geq n_0$ (下界) ($\geq$ 常數倍)
- $o(g(n))$: 滿足 $0 \leq f(n) < c g(n)$, $\forall n \geq n_0$ ($<$ 常數倍)
- $\omega(g(n))$: 滿足 $0 < c g(n) < f(n)$, $\forall n \geq n_0$ ($>$ 常數倍)

Quiz 3 Example

Finding an asymptotically tight bound for $f(n) = n^4 + 3n - 7$ by using θ -notation
Let $f(n) = n^4 + 3n - 7$, $g(n) = n^4$
 $f(n) = \theta(g(n))$ if there exist positive constants c_1, c_2 and n_0 such that
 $c_1 g(n) \leq f(n) \leq c_2 g(n)$, $\forall n \geq n_0$
 $c_1 n^4 \leq n^4 + 3n - 7 \leq c_2 n^4$ 目標 n^4
 $c_1 \leq 1 + \frac{3}{n^3} - \frac{7}{n^4}$ $1 + \frac{3}{n^3} - \frac{7}{n^4} \leq c_2$
隨 n 增長, 當 $n \rightarrow \infty$ 時
 $n=1, c_1 \leq 1+3-7 = -3$ $-3 \leq c_2$ $1 + \frac{3}{1^3} - \frac{7}{1^4}$ 會趨近於 1
 $n=2, c_1 \leq 1 + \frac{6}{16} - \frac{7}{16} = \frac{15}{16}$ $\frac{15}{16} \leq c_2$
 $n=3, c_1 \leq 1 + \frac{3}{27} - \frac{7}{81} = \frac{83}{81}$ $\frac{83}{81} \leq c_2$ s.t. $c_1 \leq 1 \leq c_2$
 $n=4, c_1 \leq 1 + \frac{3}{64} - \frac{7}{256} = \frac{261}{256}$ $\frac{261}{256} \leq c_2$ → c_1 取 1, n_0 取 3
→ c_2 取 2

當 $c_1 = 1, c_2 = 2, n_0 = 3$ 時, 存在 $n^4 \leq n^4 + 3n - 7 \leq 2n^4$, $\forall n \geq 3$
∴ 可證, $n^4 + 3n - 7 = \theta(n^4)$

Chapter 4 Divide and Conquer

三方法證明分治複雜度: 1. 置換法 Substitution Method
2. 遞迴樹法 Recursion Tree Method
計算時可先忽略細節, 上下高斯
再回頭檢查是否重要: 主定理 Master Method

1. Substitution Method (2 Steps)

- I. 依照經驗猜答案
- II. 用數學歸納法證明正確性 (找出 Constants)

Example: $T(n) = 2T(\frac{n}{2}) + n$, where $T(1) = 1$

(1) 猜測 $T(n) = O(n \lg n)$

(2) 證明 big-O notation 成立 $\rightarrow$ 找 C, n_0

Basic Step: $\rightarrow \lg 1 = 0$

$T(1) = 1 \leq C \lg 1$, $1 \leq C, \times 0 \Rightarrow C$ 不存在

$T(2) = 2T(1) + 2 = 4 \leq C, 2 \lg 2 \Rightarrow C_1 \geq 2$

$T(3) = 2T(1) + 3 = 5 \leq C, 3 \lg 3 \Rightarrow C_1 \geq 1$

$\Rightarrow C_1 = 2, n_0 = 2$

Induction Step: $\rightarrow$ 假設 $T(\frac{n}{2})$ 成立, 證明 $T(n)$ 成立

Assume $T(\frac{n}{2}) \leq C_2 \lg(\frac{n}{2})$ for $\lfloor \frac{n}{2} \rfloor$

then, $T(n) = 2T(\frac{n}{2}) + n$

$\leq 2(C_2 \lg(\frac{n}{2})) + n \rightarrow$ 從剛剛的假設置換

$\leq C_2 n \lg(\frac{n}{2}) + n = C_2 n \lg n - C_2 n \lg 2 + n$

$= C_2 n \lg n - C_2 n + n \rightarrow$ 計算結果

目標 $\leftarrow \leq C_2 \lg n \Rightarrow$ 當 $C_2 \geq 1$ 時, 恆成立

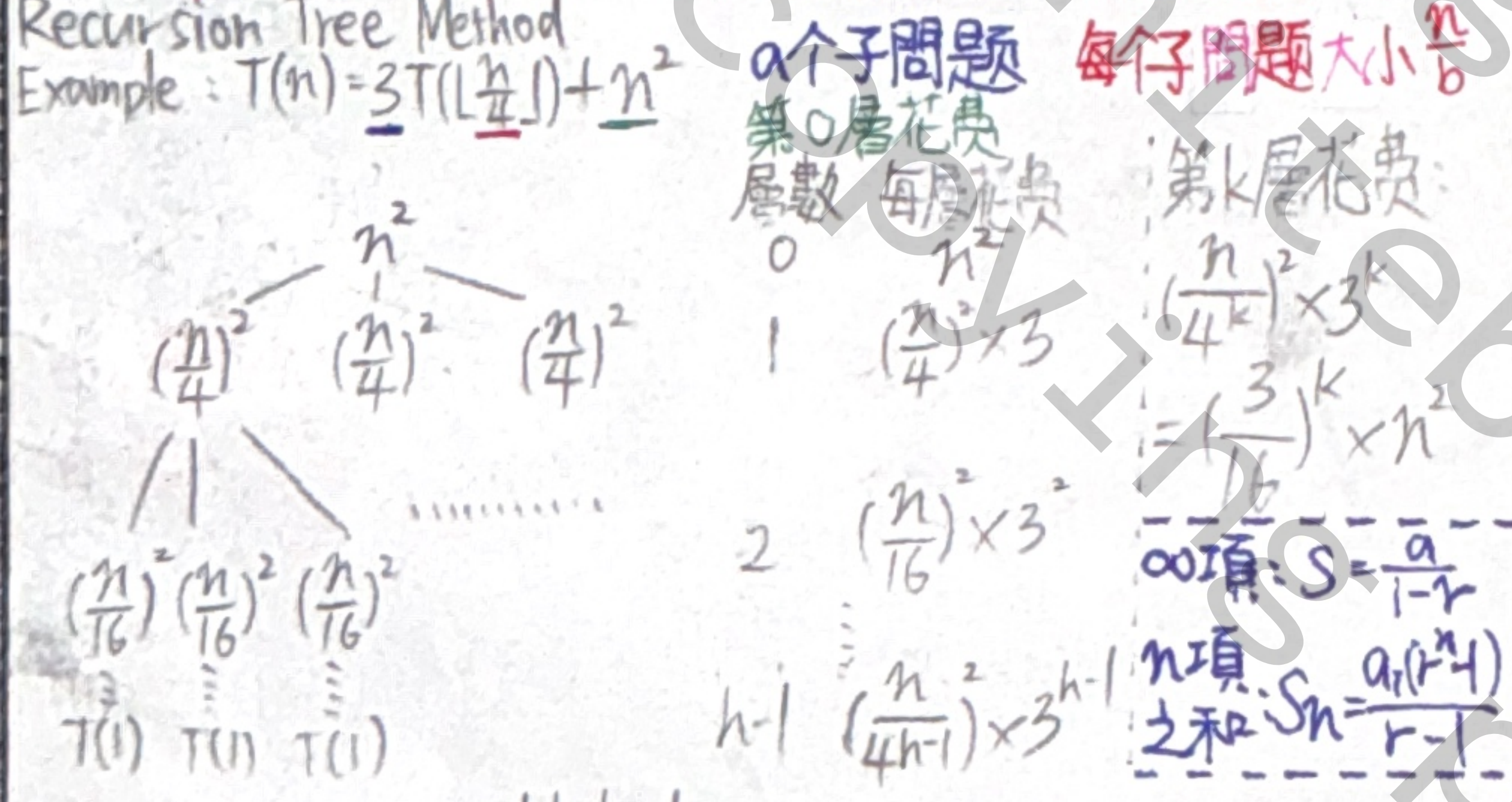
$\therefore \exists C = \max(C_1, C_2) = 2, n_0 = 1$, s.t. $T(n) \leq Cn \forall n \geq n_0$

$\rightarrow$ 如果這時找不出 C, n_0 , 就要重猜答案。

$\rightarrow$ 如果存在一個 $+n$ 消不掉, 在假設時可能要額外猜一 $-dn$ 項

$\rightarrow$ eg. $T(n) = 5T(\frac{n}{4}) + n$, 假設時不能單純 $T(n) \leq Cn^{\lg 5}$, 要

proof $\exists C, n_0$ s.t. $T(n) \leq Cn^{\lg 5} - 5n \forall n \geq n_0$ 才行!!!



let recursion tree total h level. $\frac{n}{4^{h-1}} = 1, h = \log_4 n + 1$

層數 0 1 2 ... $h-1$

問題大小 $n \frac{n}{4} \frac{n}{16} \dots \frac{n}{4^{h-1}}$

$T(n) = n^2 + \frac{3}{16}n^2 + \frac{9}{256}n^2 + \dots + (\frac{3}{16})^{\log_4 n} n^2 = \sum_{i=0}^{\log_4 n} (\frac{3}{16})^i n^2 < \sum_{i=0}^{\infty} (\frac{3}{16})^i n^2$

$\rightarrow \frac{1}{1 - \frac{3}{16}} n^2 = \frac{16}{13} n^2 \Rightarrow T(n) = O(n^2)$

Recursion Tree 只能猜大概 要再用置換法證明

Master Method

$T(n) = aT(\frac{n}{b}) + f(n) \Rightarrow T(\frac{n}{b})$ 每個子問題所耗時間

* 可以是 $\lfloor \frac{n}{b} \rfloor$ 或 $\lceil \frac{n}{b} \rceil$: D $f(n)$ 分割合併所耗時間

3種 Case: $\rightarrow$ 比較 $n^{\log_b a}$ 與 $f(n)$ 的成長速度。

Case 1 $n^{\log_b a} > f(n)$ 相差 n^e 差有正負 $\rightarrow \theta(n^{\log_b a})$

Case 2 $n^{\log_b a} = f(n)$ 不一定整數 $\rightarrow \theta(n^{\log_b a} \lg n)$

Case 3 $n^{\log_b a} < f(n)$ 相差 n^e 滿足正規條件 $\rightarrow \theta(f(n))$

正規條件 Regularity Cond.: $a f(\frac{n}{b}) \leq C f(n), C < 1$

最大子陣列分治版 Maximum-subarray $\theta(n \lg n)$

每次都分一半, 算左邊最大, 右邊最大, 跨中最大 (左半後綴 + 右半前綴), 然後合併上去。

旋轉拉表矩陣算法 Strassen's Algorithm: $T(n) = \theta(n^{\lg 7})$

Chapter 6 Heap

HeapSort 堆積排序 $O(n \lg n)$. Unstable. In-Place

本質上是一陣列, 只是將其看成一棵 Complete Binary Tree.

n 個點的 heap 高為 $\lfloor \lg n \rfloor$

max-heap: 父節點 $\geq$ 子節點。root 最大。

min-heap: 父節點 $\leq$ 子節點。root 最小。

heap 高的定義: 1 節點到某結點所經過最多的邊數。

$\rightarrow$ root 高即 heap 的高!

$\rightarrow$ 函式 max-heapify 負責維護性質

max-heapify(A, i) 會從 heap "A" 中 i 的位子(節點)一路往下維護

HeapSort 3 Steps:

I 交換 II 忽略 III 重新維護!

HeapSort(A)

Build-max-heap(A)

for $i \leftarrow \text{length}[A] \sim 2$

Swap $A[i] \leftrightarrow A[1]$ % 交換

heapsize[A] - 1 % 忽略

max-heapify(A, 1) % 維護

把當前最大一直換到最後面並忽略

這樣陣列 A 最後就會由小到大排。

* 1-Based: A 是 heap

A[1] 為 root

某節點 i 的 parent idx = $\lfloor \frac{i}{2} \rfloor$

某節點 i 的左子: $2i$

某節點 i 的右子: $2i + 1$

max-heapify(A, i) $O(\lg n)$

$l \leftarrow \text{LEFT}(i)$

$r \leftarrow \text{RIGHT}(i)$

最多 h 次交換

if $l \leq \text{heapsize}[A]$ and $A[l] > A[i]$

then largest $\leftarrow l$

else largest $\leftarrow i$

if $r \leq \text{heapsize}[A]$ and $A[r] > A[largest]$

then largest $\leftarrow r$

if largest $\neq i$

then swap $A[i] \leftrightarrow A[\text{largest}]$

max-heapify(A, largest)

* length[A]: 總共可以放幾個

* heapsize[A]: 現在放了幾個

heap insert: $O(\lg n)$ 靠 last 放

和把 ∞ 增加鍵值達成!

Chapter 7 QuickSort 快速排序 $\theta(n \lg n)$. Unstable. In-Place

與 merge sort 一樣使用 Divide and Conquer 的思路。

不同的是, 不需要額外的 combine 動作。

會選擇基準 Pivot 是 Q.S. 的核心想法。

I 從陣列中選一元素作 Pivot

II 分割 Partitioning, 遍歷陣列, $\leq$ Pivot 放左邊, $>$ 放右邊。

III 歸位 Pivot, 插入左右堆的中間。

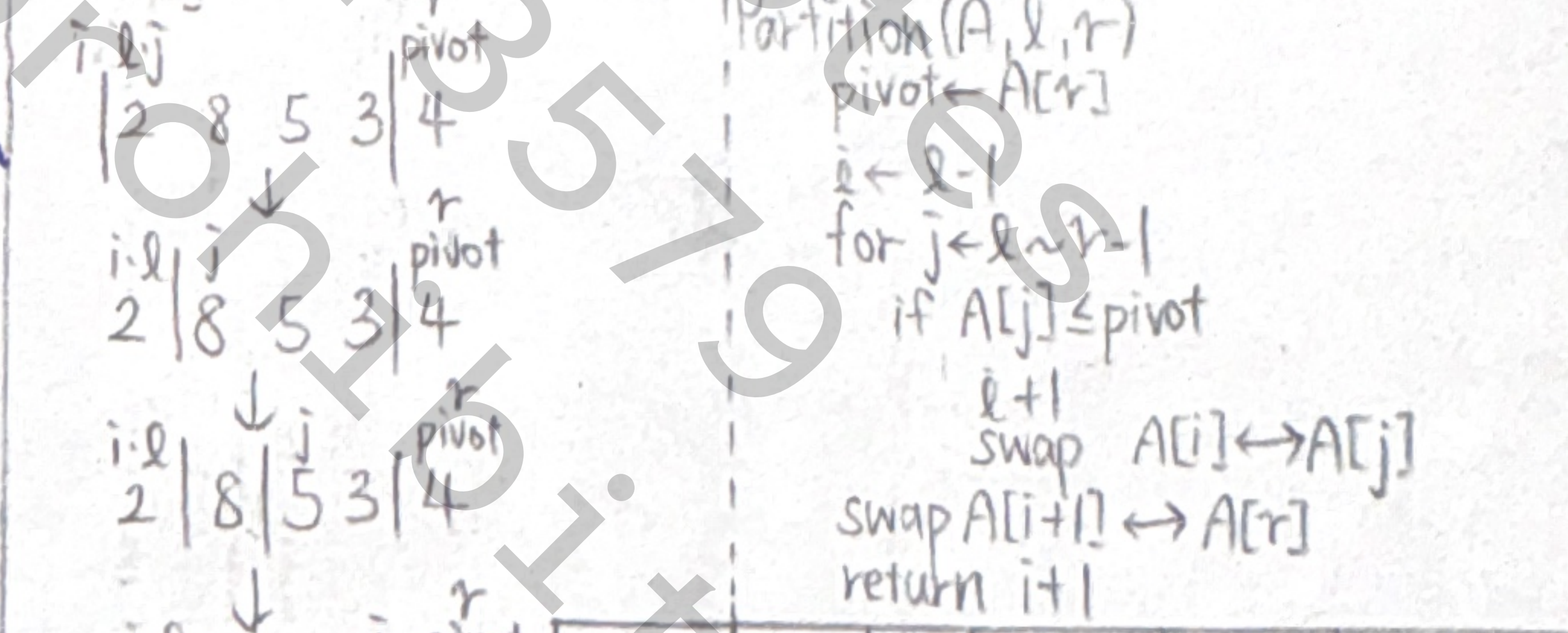
QuickSort(A, l, r)

if $l < r$

then mid $\leftarrow$ Partition(A, l, r)

QuickSort(A, l, mid-1)

QuickSort(A, mid+1, r)



	Best	Avg	Worst	Stable	In-Place
Bubble	n	n^2	n^2	✓	✓
Selection	n^2	n^2	n^2		✓
Insertion	n	n^2	n^2	✓	✓
Heap	$n \lg n$	$n \lg n$	$n \lg n$		✓
Merge	$n \lg n$	$n \lg n$	$n \lg n$	✓	
Quick	$n \lg n$	$n \lg n$	n^2		△
Counting	$n+k$	$n+k$	$n+k$	✓	
Radix	$d(n+k)$	$d(n+k)$	$d(n+k)$	✓	
Bucket	$n+k$	$n+k$	n^2	✓	

Quick Sort 雖然在原陣列交換, 但它是遞迴呼叫, call stack 會占用額外空間。

雖 in-place, 但空間複雜度非嚴格 $O(1)$!